

dsh 图像管线详解

面向 AI 应用开发者

deepseek-harness 0.1.1-rc.2

对比 opencode / cc-switch

一、为什么需要"图像管线"?

把图片发给大模型，最常见的是 **base64 内联**：

```
{
  "role": "user",
  "content": [
    { "type": "text", "text": "这张图里有什么?" },
    { "type": "image_url", "image_url": { "url": "..." } }
  ]
}
```

简单通用，但有四个致命问题：

问题	具体表现
体积膨胀	base64 膨胀 ~33%，5MB 截图变 ~6.7MB 文本；几十张图会话请求体轻松几十 MB
KV-cache 失效	服务端缓存请求前缀加速；图片字节哪怕变 1 个，整段缓存作废重算，费用延迟翻倍
无法重放	日志若存 base64 则巨大难管；重启后历史图片难以回看
无预算控制	用户一次贴 50 张 8K 截图？没有闸门直接超限报错

dsh 的答案一句话：图片存一次（内容寻址），发送时按需投影（带缓存），传输引用优先（file_id），全链路预算闸门。

二、全景架构图

① 规范化入库（提交时执行一次）	
贴图/产图 → EXIF矫正 → 去元数据 → sRGB → 长边2048px	
→ 按色彩复杂度选 PNG/WebP (85→80→75)/JPEG 逐档压缩 ≤4MiB	
→ 存 objects/<sha256>（内容寻址）→ 日志只记 sha256 引用	
▼	
② 请求投影（每次发模型时，带缓存）	
缓存键 = (attachmentId, 变换版本, 像素预算640k, 字节上限1MiB)	
命中 → 复用字节 (KV-cache 前缀稳定 ✓)	
未命中 → 按 provider 预算缩放编码 → 写入缓存	
▼	
③ 上传 wire（两种形态）	
路径A Files API优先: POST上传 → file_id (dsh-前缀)	
上传索引持久化: 同图跨会话零重复上传	
wire = {type:'file', file_id}	
路径B base64回退: wire = {type:'image_url', data:...;base64,...}	
(Files 超时/失败自动回退)	
▼	
④ 确定性预算闸门（剔除非分批）	
Files引用累计≤128MiB / 内联≤20MiB / 单图≤1MiB / 单请求≤600张	
超限按 quantum 整张剔除 —— 同样输入永远剔同一批 → 重放一致	



源码锚点: [attachment-local/src/index.ts](#) (规范化)、[request-images.ts](#) (投影缓存)、[llm-deepseek/src/file-store.ts + upload-index.ts](#) (Files 与持久化索引)、[serialize.ts](#) (wire 格式)。

三、逐层拆解

3.1 规范化入库：一张图只处理一次

贴图并回车后，第一件事不是发模型，而是把图变成规范化"对象"：

- **矫正与清洗**：EXIF 旋转实际应用；元数据与色彩配置全部剥除（对模型无意义只占体积）

- **像素标准化**：统一 8-bit sRGB/sRGBA——视觉编码器最熟悉的格式，保证后续压缩确定
- **智能压缩**：最近邻采样判断色彩复杂度后分支尝试：
 - 低色彩无透明 → PNG（调色板仅无透明时用）
 - 有透明通道 → WebP 质量 85→80→75
 - 不透明照片 → JPEG 85→80→75
 - 换档仍超 4MiB 才缩小长边重试
- **内容寻址**：最终字节算 SHA-256 存为对象 id。**同样的图永远得到同一个 id**——这是所有缓存的基石

细节：干净的 8-bit sRGB PNG/JPEG/WebP 若已在限制内会**逐字节原样通过**（零损失零开销）；GIF 只取首帧（v1 契约排除动画）。

3.2 请求投影：保护你的 KV-cache

provider 对请求前缀做缓存，但传统做法每次重新压图，字节一变前缀就失效。dsh 把编码变成**确定性纯函数**：

```
缓存键 = (attachmentId, transformVersion, pixelBudget, byteCap, encoderSettings)
```

- 同图 + 同参数 = **字节级相同输出**，结果本地缓存直接复用
- 并发读同图共享一次变换（FIFO 限流默认 2）
- 多轮对话反复引用同图时，除第一轮外每轮字节完全一致 → provider 前缀缓存持续命中

3.3 Files API 与 file_id：请求体从 30MB 缩到几 KB

DeepSeek Files API 允许先上传拿 `file_id`，聊天请求只带短字符串。rc.2 把它做成首选：

机制	实现
上传复用索引	<code>(attachmentId, variantId) → fileId, expiresAt</code> 磁盘持久化（原子写+文件锁）。第二次会话不再上传；过期前自动续传
并发去重	多请求同时传同图共享一次上传操作，不打三份
失败回退	Files 超时或配额错误自动降级 base64 内联——功能不中断只是变大
配额回收	配额满批量清理过期上传

注意：file_id 不是给模型"自己索引图片"用的。取图发生在 DeepSeek 服务端，价值纯粹在网络层——请求体从几十 MB 缩到几 KB。

3.4 预算闸门：剔除而非分批

层	默认值	保护对象
Files 引用累计	128 MiB	单请求文件引用总体积
base64 回退累计	20 MiB	回退后的请求体
单图编码	1 MiB	每张投影后的图
单请求图数	600 张	极端防滥用

超限按 quantum 步长**整张剔除**（非切片分批），算法确定性——今天被剔的图明天重放还是被剔，日志与实际请求永远一致。

3.5 read_image 坐标换算

rc.2 给 `read_image` 工具新增返回字段：**缩放后尺寸**和**坐标比例尺**。模型在缩略图上指认坐标时，工具链按比例换算回原图精确裁剪——GUI 自动化、视觉标注类插件（如 modlens）的基础设施级改进。

四、三方方案对比

对比本地主流智能体实现（opencode `src/image/image.ts`、cc-switch `src-tauri/src`）：

维度	dsh (0.1.1-rc.2)	opencode	cc-switch
定位	完整管线（存储+投影+上传）	发送前单次归一化	多客户端配置切换器，不管图像
存储	✅ sha256 内容寻址持久库，可重放	❌ 无持久化，消息存 data URL	❌ 不经手
压缩引擎	sharp/libvips, PNG/WebP/JPEG 分支	photon WASM, Lanczos3 + PNG/JPEG(80~40)	—

尺寸策略	长边 2048px, 超限逐档缩	2000×2000, 超限 ×0.75 迭代 ≤32 次	—
体积上限	归一化 4MiB + 投影 1MiB 两级	base64 后 5MiB 单级	—
传输形态	file_id 优先 + base64 回退	仅 base64 内联	—
缓存	✅ 投影缓存 + 远端上传索引, 字节级确定	❌ 每次 normalize 重算	—
预算	四层闸门, 确定性剔除	单级 5MiB, 超限报错或迭代缩	—
KV-cache 影响	同图同参=字节相同, 前缀稳定	重编码可能字节漂移, 前缀易失效	—
EXIF/色彩	矫正+剥离+sRGB 强制	未显式处理	—
动图	明确排除 (取首帧)	未显式排除	—

总结：opencode 解决"这张图能不能塞进去"（单次归一化），dsh 解决"一万张图的会话如何长期经济地跑"（存储+投影+引用三级体系）。cc-switch 是 provider 切换器，不在本问题域。

什么时候你不需要 dsh 这套？

- 单轮单图、用完即弃 → opencode 式单次归一化足够且更轻
- 只用 OpenAI 兼容接口、从不关心历史重放 → base64 直塞最简单
- 涉及多图长会话、成本敏感、审计追溯 → dsh 三级体系是刚需

五、实践建议

1. **插件开发者：**不要自己压图。用 `ctx.attachments.saveImages()` 存图拿 `sha256:` 引用；视觉工具利用 `read_image` 的坐标比例尺做精确裁剪。
2. **应用架构师：**"内容寻址 + 确定性投影 + 引用优先传输"三层模式可直接借鉴——目前开源智能体里最完整的实现。
3. **成本敏感场景：**确认 provider 支持 prefix caching，保持投影参数稳定——参数一变缓存键就变，前缀全部失效。

附：术语速查

术语	含义
base64 内联	图片编码为文本嵌入 JSON body，通用但膨胀 33%
file_id / Files API	先上传到服务商，请求携带短引用 ID
KV-cache / prefix caching	服务端缓存请求前缀计算结果，前缀字节级相同才命中
内容寻址 (sha256)	内容哈希作存储地址，天然去重不可篡改
确定性投影	同输入必同输出的编码过程，缓存正确性的前提
quantum 剔除	超预算按固定步长整张移除（非切片），保证可重放

本文基于 deepseek-harness v0.1.1-rc.2 (b150a55) 源码撰写 · 2026-08-24 · dsh-hub 项目 · 后续将逐步讲解 dsh 构建细节与大模型知识